

技术文档：Zynq-7020 多源视频实时处理与显示系统

版本对应位流 `build/system.bit` (md5 前 12 位 `cd04907e1369`) , 工具链 Vivado / Vitis 2025.2.1 , 器件 `xc7z020-clg484-2` 。上板跑通且有报告可查的条目才写进来, 没量过的直接写"没量过"。

1. 作品是什么

1.1 一句话

系统跑在一块 Zynq-7020 板上。三路视频源（网线推流、SD 卡裸帧、自研测试图卡）在 PL 里做缩放、旋转、九位控制字的算法链和同帧左右对比叠加，输出到 1024×600 的 HDMI 屏。

链路健康计数、帧率、温度、当前几何参数直接叠在屏上，同一组数也能从串口与 JTAG 读回。

1.2 功能清单与验证方式

功能	实现位置	怎么确认它真的在工作
HDMI 输入接收 (TMDS 去串化)	<code>src/rtl/hdmi/</code>	顶层台架逐像素比对；屏上出现源画面
RGMII 千兆收包 + GMII 域转换	<code>src/rtl/eth/rgmii_rx.v</code> 、 <code>gmii_to_rgmii.v</code>	<code>sim/tb_link_monitor.v</code> 与整屏台架；PL 自己数的丢帧计数为 0
UDP 视频推流接收 (512×300 RGB565)	<code>src/rtl/eth/</code> (帧重组、端口过滤)	600 帧一轮与 300 s / 9000 帧长跑，丢帧/坏帧/重复帧均为 0
ARP 请求与 ICMP ping 应答 (纯 RTL)	<code>src/rtl/eth/arp_rx.v</code> 、 <code>arp_tx.v</code> 、 <code>icmp_rx.v</code> 、 <code>icmp_tx.v</code>	主机 ping 有回包； <code>sim/tb_icmp_ping0.v</code> 等四条回归
SD 卡裸帧播放 (PS 读，不做解码)	<code>src/ps/sd_play.c</code>	串口 [STAT] 与屏上帧号；29.8–30.0 fps
自研动态测试图卡 (第三源)	<code>src/rtl/video/</code> (渐变底 + 移动块 + 帧号)	拔网线拔卡后屏不黑；长按按键切源
帧缓存与乒乓读写	DDR (AXI) + BRAM 行环	整屏台架的逐像素判据；BRAM 95.5/140 tile
缩放 0.25–2.00 (八档) + 自动呼吸 + Fit	<code>src/rtl/process/zoom/</code>	lane23 回读倍率与来源；屏上量格
旋转 + 双线性插值	<code>src/rtl/process/rotate/</code> 、 <code>bilin/</code>	<code>sim/tb_v96</code> 的映射判据 M1/M2/M4/M6；整屏台架
效果链 (gamma / 颜色 / 滤波 / 边缘 / 阈值 / 形态学)	<code>src/rtl/process/proc_pipeline.v</code> 等 11 个文件	每级一条旁路判据；固定 15 拍延迟由 <code>tb_v86</code> 实测钉住
同帧左右对比 (左原图 / 右处理图，缝位置可调)	<code>src/rtl/video/split_ctrl.v</code> + 顶层混合	串口 <code>split <n></code> 后 marker 位不变的判据；屏上看缝
OSD 叠加 (帧率、丢帧、温度、几何参数)	<code>src/rtl/video/osd_overlay.v</code>	36 行人眼验收表逐条；[TEMP] 回显与屏上三字符对账
链路健康自诊断 (计数 + 自动回落)	<code>src/rtl/eth/link_monitor.v</code> + 顶层仲裁	拔线/拔卡现场复验；AUTO 回落到可用源

功能	实现位置	怎么确认它真的在工作
QSPI 固化（断电自启）	board/scripts/make_boot_image.sh + board/tcl/flash_qspi.tcl	断电自启整条成立（Boot mode is QSPI → FPGA Done ! → SUCCESSFUL_HANDOFF）， 两格读数与凭据在 §8.3

1.3 三个创新点

1. **同一帧内的左右对比显示**：原图与处理图按列号逐像素实时切换，分割线位置是一个运行时参数（split_ctrl.v）。算法效果在同一帧里直接对照得出来，不接采集卡、不录屏。
1. **0.25–2.00 倍八档缩放、任意角度旋转与双线性插值共用一套坐标算子**：zfit 用 3 颗 DSP 算适配系数，zmap 用 8 颗做逐像素映射，旋转走逆映射、bilinear 做两次线性插值；倍率是八档定点数（25/33/50/75/100/133/150/200），另有自动呼吸档与 Fit 档，当前倍率与来源都能从 lane23 读回，屏上那一格与它同源。
- 旋转态四角有没有出屏不靠人眼，由一条两端夹逼的台架检查判掉（逐段实现见 §3.3）。
1. **一条把几何运算做到固定延迟的效果链**：六级算法、九位控制字，任意组合下整链延迟恒为 15 拍、内容偏移恒为 -4 行，坐标按每一级自己的节拍打标。整链延迟对外只有一个数 u_pipe.LATENCY，叠加算法时读口调度不用重算，某一级有没有真的生效也成了能检查的问题。

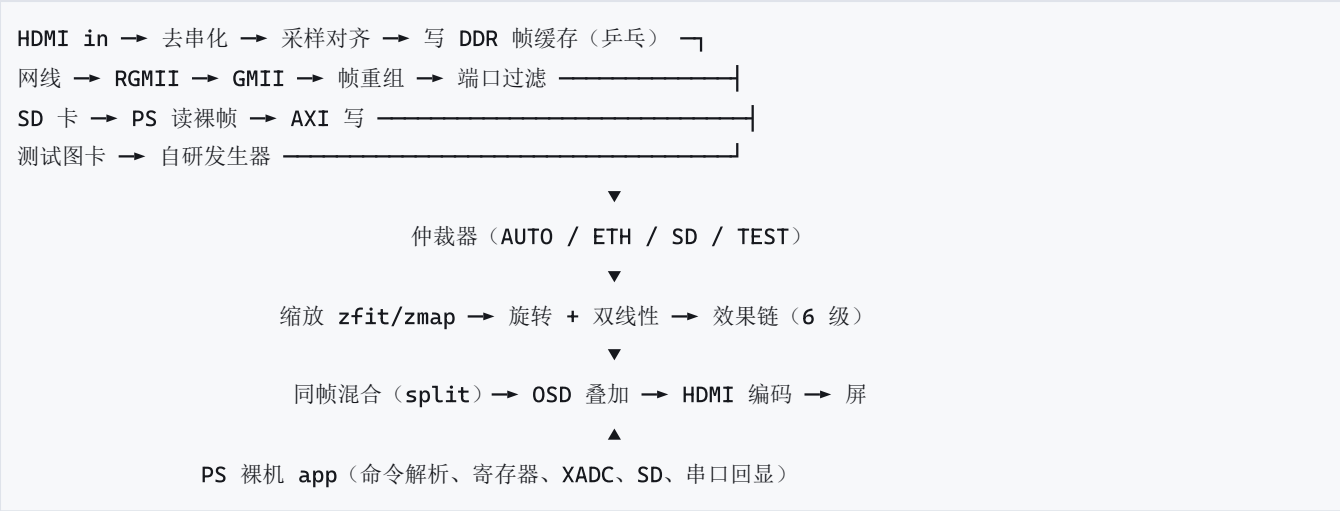
2. 系统组成

2.1 时钟与域

域	频率	周期	用途	setup WNS	hold WHS
sys_clk	50 MHz	20.000 ns	板载晶振输入、按键、慢速控制	14.876 ns	0.222 ns
clk_fpga_0	100 MHz	10.000 ns	AXI 全互连与帧缓存写侧（PS FCLK0）	1.850 ns	0.053 ns
clkout0_1	50 MHz	20.000 ns	显示读出、几何、效果链、OSD（clk_pix）	3.630 ns	0.059 ns
eth_rxc	125 MHz	8.000 ns	RGMII 收发、协议栈、仲裁	0.739 ns	0.052 ns
clkout1_1	250 MHz	4.000 ns	TMDS 串行化（OSERDESE2）	不纳检（见 §6.4）	—

全设计 51,135 个端点，setup 与 hold 的失败端点都是 0。读数出自 build/timing_summary.rpt：第 151 行是汇总行，第 154 行是 "All user specified timing constraints are met"。

2.2 数据流



2.3 规模

项	数量	来源
RTL 文件 / 行数	80 个 / 11,768 行	src/rtl/
PS 侧 C 代码行数	2,544 行	src/ps/
仿真台架	82 个	sim/ (取舍清单在 sim/README.md)
整屏逐像素判据	141 条 (140 通过 + 1 条故意留下的 C5c)	build/tb_v98_report.txt
上板人眼验收	36 行	board/verify_r87.md
一键门禁	24 项 (23 绿 / 1 条申报过的红)	build/gates.sh

2.4 用到的技术

用了什么	在哪儿	为什么是它
IDDR + IDELAYE2 抽头	src/rtl/eth/rgmii_rx.v	RGMII 是 125 MHz 双沿 4 位，只有片内双沿触发器能还原成单沿字节流；抽头用来挪采样沿找眼心
OSERDESE2 串化	src/rtl/hdmi/tmds_serializer.v	TMDS 每位要 5 倍时钟串行输出，fabric 里做 7:1 只有这根原语
MMCM2_BASE 时钟综合	src/rtl/clocks/clk_gen.v	一片板钟要同时供出像素钟、5 倍串行钟与显示读出钟，相位与倍频都得在一个地方定
BRAM 与 LUTRAM 分工	src/rtl/eth/dc_fifo.v、src/rtl/eth/axi_frame_saver64.v 等 (ram_style 显式指定)	行环要 2× 行缓冲，放 BRAM；打包 FIFO 用 LUTRAM 才不跟行环抢 BRAM 数量
DDR 帧缓存 + AXI 全互连	BD 里的 Zynq7 PS 与 src/rtl/axi/	一帧 1024×600×2 B 放不进片内，读写分离靠 AXI，写侧由 owner 信号独占
三级同步 + ASYNC_REG	src/rtl/eth/dc_fifo.v、src/rtl/eth/eth_udp_video_top.v	跨时钟域的格雷码指针必须让工具把两级放在同一只 SLICE 里，否则亚稳态窗口不受约束
AXI GPIO 作控制通道	src/rtl/top/pl_video_top.v (axi_gpio_2 等)	九位控制字与 gamma 窗口要软件随时改，而不动 BD 的中断与地址分配
XADC 读结温	src/rtl/process/effect_ctrl.v、src/ps/main.c	板上没有独立温度传感器，器件自己的系统监控是唯一的实时读数

用了什么	在哪儿	为什么是它
裸机 standalone 应用	src/ps/ (Vitis 平台在 board/vitis_platform/)	只需要 UART、SD、GPIO 与一段主循环；带操作系统的栈在这里是成本不是收益
QSPI NOR 固化启动	board/tcl/flash_qspi.tcl、 board/scripts/make_boot_image.sh	演示要断电自起，bootgen 出的镜像写进板载 32 MB flash
xsim 台架	build/sim/run_one.sh (82 个 sim/tb_*.v)	判据要能对着 RTL 反复跑；工具链自带的仿真器不需要另装环境
Node.js 与 Python 上位机	src/host/ (.mjs 26 支、 .py 3 支)	推流、串口取数、文档尺子都要一条命令能跑，不引第三方依赖
XDC 约束面	src/constraints/	时钟组、输入输出窗、不确定度与 ASYNC_REG 都写在约束里，不是靠综合猜测

3. PL 侧实现

3.1 输入与网络

RGMII 的 4 位 DDR 数据在 rgmii_rx.v 里用 IDDR 还原成 GMII 单沿流，IDELAY 的抽头值取 31。

抽头值是扫出来的：0-31 全档下 hold 每档约 +63 ps、setup 每档约 -92 ps，两线交点在 31.1，

取整数 31 (report/timing/rgmii_window_model.md ，这份不随包，在仓库历史里)。

收包链在 PL 内完成以太网帧头解析、ARP 表项、ICMP echo 应答，PS 不参与转发，

推流端口打开时板子仍然能被 ping 通。

坏帧、重复帧与被丢弃的字数由帧重组模块自己数出， link_monitor.v 用 125 MHz 域里的毫秒计数

(TC = CLK_HZ/1000) 判断链路是否静默，静默超过整定时间就请求仲裁器换源。

512×300 RGB565 限速 30 fps 跑了 600 帧一轮与 300 s (9000 帧) 长跑，

丢帧 / 坏帧 / 重复帧都是 0，实测帧率 29.99 fps、平均帧间隔 33.34 ms。

不限速打到目标 120 fps (约 36 MB/s、287 Mbps) 的 960 帧里仍未丢字，

入流能力只报下界：≥116.7 fps，过载点没顶到，不给推测值。

3.2 帧缓存与仲裁

写侧由 AXI 全互连把三路源写进 DDR 的乒乓帧缓冲，读侧按显示扫描取数。

三路源共用 bank0，同一时刻只能有一个 owner，状态就是 pl_video_top.v:152 的

M_AUTO / M_ETH / M_TEST / M_SD；owner 变化经 3 级同步送给写侧，

这一路的 CDC 检查排在门禁第 6 项，逐轮记录在仓库历史的 report/timing/ 下 (本版不随包)。

AUTO 下拔网线、拔 SD 卡都做过板级复验：能切走，插回能切回，不需要复位。

3.3 几何：缩放与旋转

缩放分两步：zfit 用 DSP 算出适配系数 (3 个 DSP48)，zmap 做逐像素坐标映射 (8 个 DSP48)。

倍率是 8 档定点数 (25/33/50/75/100/133/150/200, 即 0.25–2.00 倍), 另有自动呼吸档与 Fit 档, 当前倍率与来源都能从 lane23 读回。旋转用逆映射加双线性插值: `rotate/` 生成目标像素的源坐标, `bilin_lerp` 做两次线性插值 (6 个 DSP48), 读口按 `BILIN_ROWS = 2` 的行缓存供数。

旋转态下四角有没有出屏由一条两端夹逼的台架检查判掉, 不靠人眼。

DSP 合计 19 个, 按实例分是 `u_lerp 6 + u_blur 1 + u_split_ctrl 1 + u_zfit 3 + u_zmap 8`。

gamma 一级不占 DSP, 它用 LUTRAM 查表, 不做乘法。

3.4 效果链

`proc_pipeline.v` 的六级顺序是 gamma、颜色、滤波、边缘、阈值、形态学。

控制字 9 位 (`gpio_cfg[8:0]`), 一位一个算法), 级 5 的腐蚀与膨胀两位互斥,

同时置 1 时两个都不做 (`proc_pipeline.v:59–60`)。

四个窗口级各自维护自己的列标签, 抽头号按该级之前累积的拍数取: blur 用 2、sharp 用 5、

sobel 用 8、morph 用 12, 所以任意组合下整链延迟固定 15 拍、内容偏移固定 -4 行。

顶层不许自己另写一个延迟数, 只取 `u_pipe.LATENCY`; `tb_v86` 实测 `de_in → de_out` 再与它比, 任何一处漂移都会有测试变红。

3.5 同帧对比与 OSD

`split_ctrl.v` 输出一个按百分比定位的缝, 顶层逐像素决定这一列取原图还是取处理图,

左右两半因此来自同一帧的同一时刻。串口 `split <n>` 之后 marker 位不变,

这条检查防的是参数改了而通路没跟着改。

OSD 叠四行信息: 源与分辨率、帧率与丢帧、几何参数、温度与状态。

字模索引在 1024×600 有效区整屏走一遍会越出数组上界 190,464 拍,

这个数划出了可用样本的下限, 屏上没现象时能区分“没机会触发”与“真的被门住”。

温度对账有三路: 驱动读数、屏上三字符、写进 PL 的 gpio 值。

4. PS 侧实现

裸机 app (`src/ps/main.c` 等, 2,544 行) 跑在 `ps7_cortexa9_0`, BSP 用仓库内的

`vitis/platform/.../bsp`, 做五件事: 串口 115200-8N1 命令解析、寄存器整字写、

SD 卡 FAT32 目录读取与裸帧推送、XADC 采样、状态回显。

控制字只有一种写法: PS 一次整字写 `gpio_cfg`, PL 侧统一打三级同步再分发,

单个位不许各改各的 (`effect_ctrl.v:31` 把这条写成注释)。

两个运行时开关各占一个空位, 各走一条独立同步链: `gpio_o[19]` 是双线性开关、

`gpio_o[20]` 是 OSD 总开关 (反相, 复位值 = 有 OSD)。

倍率、来源、手动/自动标志、bilin 状态都从 lane23 读回，串口与 JTAG 两条路都读得到，开关在硬件里到底是什么状态由读数说话，不靠看屏幕。

5. 软硬件划分依据

任务	放在	原因
TMD5 去串化、RGMII 收发、帧重组	PL	125 MHz DDR 采样与逐字校验，软件来不及
ARP/ICMP 应答	PL	推流占满 CPU 通道时仍要能 ping 通
逐像素效果链、缩放、旋转、双线性	PL	50 MHz 像素率下每像素 20 ns 预算
同帧混合与 OSD 叠加	PL	必须与扫描同步，不能等软件
FAT32 解析、SD 读、串口命令、XADC	PS	事务性、低频、需要文件系统
参数整定与演示脚本	PS + 上位机	人要能改，且要留日志

划分按每像素预算定，与哪边写得方便无关：像素率 50 MHz 时一像素 20 ns，任何进软件的逐像素操作都会掉帧，链路上因此没有一处逐像素的软件参与。

6. 时序与状态

6.1 收敛方法

- 时钟树分两支：clk_fpga_0 100 MHz 由 PS7 的 FCLK_CLK0 给（build/report/clock_util.rpt 的 g0 行，驱动脚是 .../FCLK_CLK_0_BUFG/0）；显示与算法那几支由 PL 侧一只 MMCME2_BASE 产生（src/rtl/clocks/clk_gen.v:15，喂进来的是 sys_clk 50 MHz，见 src/rtl/top/pl_video_top.v:128），clkout0_1 = clk_pix 50 MHz、clkout1_1 = clk_pix5x 250 MHz 都出自这只 MMCM；
- RGMII 收口的 IDDR 与延迟抽头按 §3.1 的实测交点定；
- 跨域一律 3 级同步 + ASYNC_REG，格雷码指针链单独钉（dc_fifo 四颗 FF）；
- 时钟分组显式声明（src/constraints/clock_groups_impl.xdc），异步组之外的跨域路才受检查；
- 关键路径按域逐个量，不只看全局最差那条：每域前 12 档都取过端点（build/evidence/r124_tiers_ladder.txt）。

6.2 关键路径在哪

eth_rxc 的 0.739 ns 落在 u_icmp_tx/ip_head_reg[4][16] → check_buffer_reg[19]/D 上，11 级逻辑含 6 个 CARRY4，布线占 58.4 %。同一只锥贡献前 3 档（0.739 / 0.873 / 0.961），第 4–12 档全部是 u_rx_par/p_eof_reg → u_reasm/rows_hit_reg[*]/CE，4 级逻辑、布线占 85 %。这一档的瓶颈是使能广播，校验和锥整族搬走的上界因此只有 +0.278 ns（1.017 – 0.739）。clk_fpga_0 的 1.850 ns 前 12 档全部同起点 u_pl/u_arb/owner_eth_reg/C，终点是帧缓存 BRAM 的写使能与地址脚，逻辑只有 1–4 级、布线占 88.8–93.6 %。这个域没有算法锥可拆，能动的是这只高扇出旗标的复制与摆放。

6.3 试过并被实测否掉的优化

做法	结果	依据
校验和摊到每拍一项	eth_rxc 0.739→0.691、clk_fpga_0 -0.123、sys_clk 14.876→14.068，判负回退	report/log/issues.md（台账不随包） #380
编译期常量抽出发一拍	更差：常量并入同一拍使加法树变深，端点只从 4835 减到 4819	#382
强制复制 239 引脚广播网	本域赢 1 格、另三域跌 4 格（含最紧两格），判负	report/timing/round_r117.md（不随包）
Pblock 约束摆放	载链逃出边界报 Place 30-439，且未换来收益	#186
BRAM 换 setup	+0.182 ns 相对 +0.516 不够，砍掉 FF 后仍剩 4 级 ⇒ 拥塞主导	#122
RGMII 输入窗挂上	挂窗即 -2.885 ns，退回候选件	report/timing/rgmii_window_model.md（这份不随包，在仓库历史里）

结论写在 report/timing/eth_rxc_partition_options.md（不随包）第 8 节：本版不再安排时序优化轮。

当前是零违例、约束全满足的状态，再动它换到的是余量，换不到正确性。

6.4 仍然欠的账（不因为停手而消失）

- 四个域都没有 set_clock_uncertainty 的 setup 项，只有 eth_rxc 有一条自加的 hold 带（src/constraints/rk_zynq7020.xdc:50），四列 hold 用的不是同一套测量条件；
- 屏那一路 6 个 TMDS 输出口没有对外时序结论：HDMI 的“钟↔数据 ≤0.20 Tcharacter”是展宽不是捕获窗，按 set_output_delay 挂上去量出来是 -5.408 / -1.923 ns 整批红，判 DECLINE（\$6.3 同源）；
- 异步时钟组把 4 条跨域路从检查范围里拿走了，set_max_delay -datapath_only 那份候选约束从未进默认集；
- 端到端时延只有出口（OSD 的 Latency lane）没有值，指标表那一格写 未报；33.34 ms 是 PL 内的入流帧间隔，

原先挂在“端到端时延”名下且轮次写成“一轮 600 帧”，本轮按凭据改名并把轮次写成 9000 帧 / 300 s（台账 #416）。

7. 验证与实测

7.1 功能正确性怎么确认

功能正确性分三层，各层互不替代。

- 模块级是 82 个 testbench（sim/），每条检查项都要求能红：靠空集通过的检查项本身算失败。
- 5 支变异对照（build/sim/mut_control.sh）钉住一次变异只许红它声称红的那一条。
- 整屏台架是 tb_v98，跑一遍全设计得 141 条逐像素判据（140 通过 + 1 条申报过的 C5c），报告头部带编译时戳与 RTL/台架 md5。
- 板级这一层只能由人做：36 行人眼验收（board/verify_r87.md），每行给命令、预期现象，以及这一行不通意味着什么。屏幕现象由看屏幕的人判，机器检查不代替眼睛。

7.2 性能与资源

指标	数值	条件
显示	1024×600 @ 59.5 Hz	像素钟 50 MHz, H_TOTAL 1344 / V_TOTAL 625
入流零丢包	0 丢帧 / 0 坏帧	512×300@30 fps, 限速 15 MB/s, 600 帧一轮
长跑	0 丢帧 / 0 重复帧	300 s、9000 帧, 实测 29.99 fps
入流下界	≥116.7 fps 不丢字	不限速、目标 120 fps、960 帧; 过载点未顶到
入流帧间隔 (PL 内)	33.34 ms (min 20 / max 51)	PL 收包链相邻两帧之间隔; 9000 帧 / 300 s 那一轮; 不含编码、网线与交换机排队
SD 播放	29.8–30.0 fps	预转换 RGB565, 板上不解码
Slice LUT	14,154 (26.61 %)	实现后报告
Slice 寄存器	8,188 (7.70 %)	同上
BRAM tile	95.5 / 140 (68.21 %)	帧缓存乒乓 + 行环是主要去向
DSP48	19 / 220 (8.64 %)	逐实例见 §3.3
动态功耗	2.213 W	工具估算, 置信度 Low (无仿真活动文件输入)
结温	52.6 °C (估算) / 61.2–61.7 °C (板读 XADC)	估算是工具值; 板读是六次读数极值

7.3 优化前后的对比

本版绝对值取自 `build/utilization.rpt`, 与 `data/metrics.csv` 同源, 指标表每行都点名它的证据文件:

Slice LUT 14,154 (26.61 %)、寄存器 8,188 (7.70 %)、BRAM 95.5/140 tile (68.21 %)、DSP48 19/220 (8.64 %)。

逐轮增减只保留有件可查的三笔, 出处是 `data/metrics.csv` 的资源行与 `build/evidence/` 的逐层件:

变化	量	这一格为什么变
收包链把 <code>pay_start</code> / <code>pay_end</code> 两个 16 位界提前一拍寄存	寄存器 +22	用触发器换掉 <code>p_good</code> 锥里的一级加法; 与"加寄存器换走线"同族
行覆盖使能独热化	LUT -243 (其中 -66 能归到这一刀, 其余没有归属)	没有归属的那部分不念成收益
两只按键去抖的武装门 + 校验和累加器 32→20	LUT +35 (+66 与 -31 相减)、寄存器 +34 (+46 与 -12)	净账由两份逐层件相减闭合

帧缓存与行环是 BRAM 占用的大头。打包 FIFO 从触发器改成 LUTRAM 释放了约 5 万个 FF, 代价是 BRAM 一度顶到 98.93 %, 随后回落到本版的 68.21 %。

余量侧的逐轮读数 (含 `eth_rxc` 从 0.445 ns 走到 0.739 ns 那两刀) 在仓库历史里那份 `report/timing/` 逐轮页中 (本版不随包), 这里不重复列; 那些数要摆在同一套测量条件下才可比。

WNS 的绝对差不算收益也不算损失, 只有同一域自己的关键路径催动了才算, 且别的域不许变差。

上表最后一列因此写"为什么变", 不写"变好了多少"。

8. 复现

8.1 从零建工程并跑门禁

`board/` 里那份上板工程由两支脚本落位并验证: `board/tcl/stage_board_projects.tcl`、

board/scripts/stage_vitis_platform.sh , 检查项与实跑读数记在 board/measured/stage_2026-10-05.txt 。

```
# Vivado: 建工程 → 综合 → 实现 → 位流 (约 2 小时, 含时序报告)
<DV>/bin/vivado.bat -mode batch -source build/tcl/build_system_axigpio.tcl
# 一键门禁 (24 项)
bash build/gates.sh
# 整屏台架 (约 108 分钟)
bash build/sim/run_one.sh tb_v98
```

8.2 上板 (JTAG)

```
export VP_XSDB="<Vivado>/../Vitis/bin/xsdb.bat"
bash build/board_verify.sh --geom --battery      # 串口回显与几何判据, 留原始日志
```

8.3 固化到 QSPI (断电自启)

板载 32 MB QSPI NOR (丝印 W25Q256FV, 3.3 V, 挂在 PS 的 MIO1/2/3/4/5/6 = CS/DQ0/DQ1/DQ2/DQ3/CLK) 。

```
export VP_VIVADO_BIN="<Vivado>/bin"
bash board/scripts/make_boot_image.sh          # FSBL + bit + app → board/flash/BOOT.bin (生成件, 不随包)
VP_QSPI_PART="mx25l25645g-qspi-x4-single" \
  "<Vivado>/bin/vivado.bat" -mode batch -source board/tcl/flash_qspi.tcl
```

三个坑是实测踩到的。

1. bootgen 的 bif: the_design: + 花括号里逐行一个文件、不写逗号, **但 FSBL 那一行必须带

[bootloader] **。缺这个属性时 bootgen 照样打印 Bootimage generated successfully ,

产出的镜像头里分区表偏移那三个字段 (IHT+0x10 / +0x14 / +0x20) 却全是 0, 启动 ROM 找不到

分区, 冷上电的表现是串口一个字节都不出、DONE 不亮。依据两条:

bootgen -bif_help bootloader 写明该属性支持 zynq; 厂商镜像同三个字段是

0x000001700 / 0x00018008 / 0x00018008 , 补上属性之后本工程这份变成同一形状 (数值差来自 FSBL 大小) 。

1. 部件名不能照板子丝印填: 丝印是 W25Q256FV, 可 Vivado 部件库里 w25q256jw* 那一支的

COMPATIBLE_PARTS 只列 zynqplus, 配 zynq7000 会在擦写那一步判 Labtoolstcl 44-655 ;

要看的是这张兼容表, 不是型号后缀。同为 32 MB / x4 的 macronix 档实测擦写与回读校验都通过。

1. hw_cfgmem 的属性: 2025.2.1 没有 PROGRAM.BBF_FILE、START_ADDRESS 与 STATUS

(写死会 17-142、17-54 中断) , 而 PROGRAM.ZYNQ_FSBL 又必须设, 缺它报 [Labtools 27-3203] 。

擦写另有两条规矩: 先把启动模式拨到 JTAG 再写 (QSPI 档下工具报 [Xicom 50-100] ,

这时写进去的东西它报成功也不可信) , 以及 program_flash -erase_all 在这颗片上失败、扇区擦可用。

写完拨回 QSPI、断电重上, 两格都量过了:

断电自启	实测读数	凭据
位流 (PL 配置)	成立: 串口出 FSBL 横幅、Boot mode is QSPI、QSPI is in 4-bit mode, 随后 DMA Done !、FPGA Done !	board/measured/qspi_coldboot_selfboot_ok_2026-10-06.txt
应用 (PS 侧 src/ps)	成立: SUCCESSFUL_HANDOFF 之后 [CFG] axi_gpio_2 @41220000 ok、[CFG] gamma window @41220008 ok、[TEMP] PS-XADC @F8007100 ok, 接着 [CTRL] ... src=1, SD 自动播起片, ETH 那一路也出画面	同一份回显 (4,895 字节)

应用这一格第一次量的时候并不成立: 串口停在 Handoff Address: 0x00000000 →

No Execution Address JTAG handoff (那一份回显留在

board/measured/qspi_coldboot_2026-10-06_fsbl_from_flash.txt, 它同时是“头部修好了、位流确实从 flash 起来了”的凭据)。

读平台自带的 FSBL 源码能对上—— image_mover.c 的

LoadBootImage() 遇到“PS 分区且加载地址为 0”直接跳出分区循环 (它自己的注释写着

loop will break on PS load address zero), 交接地址因此从未被赋值; 而 readelf -l 实测

fsbl.elf 与本应用的第一个 LOAD 段都是 VirtAddr 0x00000000 (OCM 低 192 KB 别名窗),

真把应用装到 0, 就是 FSBL 一边执行一边覆盖自己。所以修法只有一个方向: 应用链到 DDR。

- src/ps/lscript_ocm.ld 的 MEMORY 第一段 ORIGIN 从 0x00000000 改成 0x00200000。

这个基址是量出来的: PL 的三个 bank 在 0x10000000 / 0x10080000 / 0x10100000

(system_top.v、eth_udp_video_top.v、main.c 的 FRAME_ADDR), 中间这一大片全仓零引用;

上界受 translation_table.S 的映射窗限制 (只把 [0x00100000, 0x3FFFFFFF] 当 cacheable)。

第二段 0xFFFF0000 (各模式栈) 不动。

- 两支构建脚本 (build/ps_app.mjs 与零依赖的 build/build_ps_app.py) 里那条旧判据

"_vector_table 必须在 0x0, 否则 FATAL"本身就是这个坑的守门人, 换成三条等值判据:

入口 = _boot、_vector_table = 第一个 LOAD 的 VirtAddr、且该基址落在

[0x00100000, 0x3FEF0000] 且不为 0; readelf -l 取不到 LOAD 行时判"这条没跑, 不算通过"。

- 位流与 C 语句一个字没动; 重编后的 ELF 是 build/ps_app.elf md5 57fa442a7eaf,

重打的 BOOT.bin 是 2,435,868 字节 / md5 f62b1b8cc3bd (三份输入里位流仍是 cd04907e1369)。

在这块自启的板子上跑了机器能判的那一半: bash build/board_verify.sh --battery --geom --round=r126

⇒ RESULT board_verify PASS (判红的步骤: 0), 串口电池 105 条命令 98.2 s 全过、跑完回到初态且未态

= 演示默认档, 几何那一跳退出码 0, 温度格三方对账自治。原始回显在 build/evidence/r126_serial_raw.txt 与 build/evidence/verify_1006_1944.txt (这两份不随包, 读数已抄在上面)。

所以演示可以直接断电上电, §8.2 那三步 JTAG 加载仍然可用——换了 app 不想重刷 PL 时就用它。

写入的逐条结果记在 `board/measured/flash_qspi_2026-10-05.txt`，里面有镜像与三份输入的 md5、Erase/Program/Verify 三行成功、耗时 137 s。PROGRAM.VERIFY=1（或 `program_flash -verify`）那一步是从 flash 读回逐字节比对，Verify successful 只证明片上内容与 BOOT.bin 一致；

"断电后能自己起来"要另一次上电才算量过——上面那两格就是这么来的。

对照实验也留在包里：把厂商那份 BOOT.BIN 写进同一颗 flash 再冷上电，DONE 亮、串口打出 U-Boot（`board/measured/qspi_vendor_control_2026-10-06.txt`），说明板子、拨码、启动 ROM、这颗 flash 和写入流程都没问题，问题只可能在镜像本身。

9. 文档与目录

目录	内容
src/	RTL（rtl/）、PS 裸机（ps/）、上位机工具（host/）、约束（constraints/）
sim/	82 个台架与运行脚本，sim/README.md 是取舍清单
build/	构建脚本与综合/实现报告、门禁、证据件
board/	上板工程（Vivado 工程 + Vitis 平台）、运行脚本、实测输出、QSPI 固化脚本
data/	测试向量、黄金参考、metrics.csv 指标表
skills/	技能包（49 条，八节外壳）
report/	设计报告分章、协作记录、限制与欠账

指标表 `data/metrics.csv` 是数字的唯一出口，每行都点名它的证据文件。

这里的数字与它同源，两处不一致时以证据件为准。